PUCRS | ESCOLA POLITÉCNICA

Disciplina: Fundamentos de Processamento Paralelo e Distribuído (98713-04)

Semestre: 2026/2
Professor: Marcelo Veiga Neves

Trabalho T1

Problemas Clássicos de Concorrência: Jantar dos Filósofos e Produtor/Consumidor em Go

Grupo B

Leonardo Noronha Dornelles
Nícolas Michels
Pedro Fialho
Raul Antonio da Silveira

Porto Alegre
Outubro de 2026

1 Introdução

Este relatório descreve nossas soluções em Go para o Jantar dos Filósofos e para o Produtor/Consumidor com buffer limitado. Para cada problema explicamos as decisões de projeto, analisamos onde pode haver *deadlock* e *starvation* e discutimos os dados de execução coletados. Os dois programas são independentes, ficam nas pastas `filosofos/` e `produtor-consumidor/` e compartilham o `go.mod` da raiz. O código e as instruções de execução estão em `https://github.com/leonardondornelles/T1-FFPD`.

2 Problema 1: O Jantar dos Filósofos

2.1 Solução adotada

Cada garfo é um canal com buffer de tamanho 1 (`chan struct{}`). Pegar o garfo é enviar um valor no canal, o que bloqueia se ele já estiver ocupado, e soltar é receber esse valor de volta. Assim a exclusão mútua sobre o garfo vem do próprio canal, sem precisar de *mutex*. O envio fica dentro de um `select` junto com um canal de cancelamento `done`, para que um filósofo bloqueado possa desistir se a mesa for cancelada.

O programa recebe o número de filósofos (`-n`), o número de refeições (`-r`) e a estratégia (`-strategy`). Os tempos de pensar e de comer são sorteados entre 5 e 15 ms, com um gerador por filósofo. Usamos tempos variáveis porque, com tempos fixos, todos os filósofos andariam sincronizados e os dados de *fairness* não diriam muito sobre as estratégias. As três versões são:

- `deadlock` (versão base): todos pegam primeiro o garfo da esquerda, esperam 20 ms e tentam o da direita. Como o tempo de pensar varia no máximo 10 ms entre os filósofos, todos já seguram o garfo esquerdo quando o primeiro tenta o direito, e a mesa trava logo na primeira rodada.
- `hierarchy` (hierarquia de recursos): todos pegam primeiro o garfo de menor índice. Na prática só o filósofo $N-1$ muda de ordem, porque seus garfos são o $N-1$ e o 0.
- `conductor` (limite de lugares à mesa): um canal com capacidade $N-1$ funciona como semáforo de contagem, e no máximo $N-1$ filósofos disputam garfos ao mesmo tempo.

2.2 Análise de deadlock e starvation

Pelas condições de Coffman, o *deadlock* só acontece se as quatro valerem ao mesmo tempo. Aqui há **exclusão mútua** (um garfo serve a um filósofo por vez), **posse e espera** (o filósofo segura um garfo enquanto espera o outro), **não preempção** (ninguém tira o garfo de outro) e pode haver **espera circular** (cada um espera o garfo que está com o vizinho).

Na versão `deadlock` as quatro condições acontecem e nenhum filósofo avança. Para o programa não ficar travado para sempre, uma *goroutine* vigia (*watchdog*) acompanha um contador atômico de refeições. Se ele não muda por 2 s, o *watchdog* fecha `done`, todos os filósofos desistem e o programa informa o *deadlock*. Ele não impede o *deadlock*, só detecta e encerra a mesa. Nessa versão a *starvation* é total, porque ninguém volta a comer.

A `hierarchy` elimina a **espera circular**. Se todos pegam os garfos em ordem crescente de índice, quem segura o garfo de maior índice nunca espera por um de índice menor, então não tem como fechar um ciclo. A `conductor` também elimina a espera circular, mas por contagem: o ciclo precisaria dos N filósofos segurando um garfo cada, e só $N-1$ estão na mesa. Com $N-1$ filósofos e N garfos, pelo menos um consegue os dois garfos, come e libera os recursos. A posse e espera continua existindo na `conductor`, o que aparece nos dados a seguir.

Nenhuma das duas estratégias garante formalmente ausência de *starvation*, já que o Go não promete ordem justa no escalonamento das *goroutines* nem no desbloqueio dos canais. Nos testes, porém, todos os filósofos sempre completaram as R refeições.

2.3 Análise de fairness

A Tabela 1 mostra a espera pelos garfos com $N = 5$ e $R = 200$. Cada valor é a média de dez execuções sem `-race`, que distorceria os tempos, em Go 1.27.0, Windows, AMD Ryzen 5 5600G. Como cada filósofo faz exatamente R rodadas, todos fizeram 200 refeições em todas as execuções, e por isso a *fairness* aparece no tempo de espera, não na contagem de refeições.

Tabela 1: Espera pelos garfos por filósofo ($N = 5$, $R = 200$, todos com 200 refeições). "Média" é a média de 10 execuções; "Máxima" é a maior espera observada.

	`hierarchy`		`conductor`	
Filósofo	Média (ms)	Máxima (ms)	Média (ms)	Máxima (ms)
0	8,55	45,8	20,84	48,2
1	6,21	31,2	19,86	50,9
2	5,74	30,9	19,80	50,0
3	5,44	23,3	20,31	49,5
4	7,53	43,5	19,70	47,4
Média geral	**6,69**		**20,10**	
Diferença maior/menor	**3,11**		**1,14**	

Os dados mostram uma troca entre velocidade e justiça. A `hierarchy` espera bem menos (6,69 ms contra 20,10 ms), mas de forma desigual: os filósofos 0 e 4 esperam mais que os outros, e a espera máxima do filósofo 0 (45,8 ms) é quase o dobro da do filósofo 3 (23,3 ms). Isso se repetiu em todas as execuções. A causa é a própria ordenação: os filósofos 0 e 4 disputam o garfo 0 como *primeiro* garfo. Na `conductor` todos são tratados igual, e a diferença entre o filósofo que mais espera e o que menos espera cai para 1,14 ms.

A espera maior da `conductor` tem duas explicações. A medição inclui o tempo esperando um lugar na mesa. Além disso, como a posse e espera continua, os filósofos sentados podem formar uma fila em que cada um segura o garfo esquerdo esperando o do vizinho, e só o último da fila come. Na `hierarchy` os filósofos 0 e 4 disputam o garfo 0 de mãos vazias, o que corta essas filas. Concluímos que a `conductor` é mais justa e a `hierarchy` é mais rápida.

2.4 Encerramento ordenado

Cada filósofo é uma *goroutine* que chama `wg.Done()` ao terminar suas R rodadas ou ao ser cancelado. Toda espera por recurso, inclusive o lugar na mesa da `conductor`, é um `select` que também escuta `done`, então nenhuma espera é infinita. A *goroutine* principal espera os filósofos (`wg.Wait()`), fecha o canal `finished` para avisar o *watchdog* e espera ele terminar também. Só depois imprime os resultados. Na versão `deadlock` o caminho é o mesmo, pois o fechamento de `done` libera todos os filósofos bloqueados. Parâmetros inválidos são recusados antes de criar qualquer *goroutine*.

3 Problema 2: Produtor/Consumidor com Buffer Limitado

3.1 Solução adotada

As duas versões ficam no mesmo programa, escolhidas por `-mode channel` ou `-mode semaphore`, e implementam a mesma interface de fila. Produtores, consumidores, encerramento, verificação e medições são o mesmo código nos dois modos. Assim, a diferença de desempenho entre eles vem só da implementação do buffer.

Versão com channel. O buffer é um `chan item` de capacidade K, e o bloqueio com buffer cheio ou vazio vem do próprio canal.

Versão com semáforos. O buffer é um vetor circular de tamanho K com índices `in` e `out`, seguindo o algoritmo clássico visto em aula. Os semáforos de contagem `notFull` (começa com K) e `notEmpty` (começa com 0) são canais de permissões. O produtor faz `P(notFull)`, trava `mutexIn`, escreve na posição `in`, destrava e faz `V(notEmpty)`. O consumidor faz o caminho inverso com `notEmpty`, `mutexOut` e `notFull`.

Usamos **dois mutexes separados**, como pede o enunciado ao falar em exclusão mútua nas posições de escrita *e* de leitura. O `mutexIn` impede que dois produtores escrevam na mesma posição, e o `mutexOut` impede que dois consumidores retirem o mesmo item. Produtor e consumidor nunca disputam o mesmo lock. Quem impede que os dois mexam na mesma posição são os semáforos: `notEmpty` só libera posições já escritas e `notFull` só libera posições já lidas. Pelo modelo de memória do Go, o envio em um canal acontece antes do recebimento correspondente; somado à ordem imposta pelo `mutexIn`, isso garante que o consumidor enxerga o item escrito pelo produtor, o que o `-race` confirma.

A integridade é conferida por uma verificação por identificador nas duas versões, que também detectaria um item duplicado, e, na versão com semáforos, por contadores atômicos dentro do buffer. O programa imprime `totalProduzido == totalConsumido` e termina com erro se algo não bater.

Consumo com timeout. O consumidor 0 lê com `select` e `time.After`. Quando o tempo acaba, ele registra o timeout no log e volta a esperar. Ele não encerra, porque um timeout só indica que a produção está lenta, não que acabou.

3.2 Análise de deadlock e starvation

Na versão com semáforos, a permissão é sempre pega antes do mutex, e quem segura um mutex não espera por mais nada: só escreve ou lê uma posição e o libera. Um produtor pode segurar uma permissão enquanto espera o `mutexIn` (há posse e espera), mas, como a ordem de aquisição é sempre a mesma, não se forma ciclo: a condição eliminada é a **espera circular**. Com o buffer cheio os consumidores conseguem esvaziá-lo, e com ele vazio os produtores conseguem enchê-lo, então não há *deadlock*. Na versão com channel, o canal é fechado depois que todos os produtores terminam, e nenhum consumidor fica esperando para sempre.

Não ter *deadlock* não garante divisão justa entre consumidores. Os dados abaixo mostram que o consumidor 0 processa menos itens, mas ele nunca para de progredir, então não é *starvation*. Os produtores também não sofrem *starvation*: em todas as execuções cada um entregou todos os seus itens.

3.3 Análise do efeito do buffer

A Tabela 2 foi gerada com o modo `-bench`, com $P = 4$, $C = 4$, 500.000 itens por execução e oito repetições por combinação, no mesmo ambiente da seção anterior. *Throughput* e ocupação foram medidos em execuções separadas, porque medir a ocupação exige um lock e uma

leitura do relógio a cada operação, o que deixaria o programa mais lento. A ocupação média é calculada pelo tempo que o buffer passou em cada nível, e não por amostras periódicas, que no Windows eram poucas demais.

Tabela 2: Efeito de K sobre throughput e ocupação ($P = 4$, $C = 4$, 500.000 itens). A última coluna mostra os itens consumidos por cada consumidor, em milhares (média das oito repetições).

Versão	K	Throughput (itens/s)	Desvio	Ocup. média	Pico	C0/C1/C2/C3 (mil)
`channel`	1	2.546.049	3,8%	0,50	1	89/136/137/138
`channel`	10	3.848.379	2,0%	5,32	10	70/142/144/144
`channel`	100	6.244.948	6,7%	54,22	100	55/147/150/148
`semaphore`	1	1.574.680	2,2%	0,41	1	124/125/125/125
`semaphore`	10	2.629.038	6,0%	5,64	10	96/134/135/135
`semaphore`	100	3.910.636	5,3%	63,49	100	93/136/136/136

Efeito de K. De $K = 1$ para $K = 100$ o *throughput* cresce cerca de 2,5 vezes nas duas versões. Com $K = 1$, cada item precisa de um produtor e um consumidor prontos ao mesmo tempo, e o buffer fica quase sempre vazio (ocupação média perto de 0,5). Com $K = 100$ a ocupação média sobe para 54 a 63 e o pico chega a K: o buffer passa a absorver as diferenças de ritmo e os produtores não precisam esperar a cada item. Ou seja, um K maior desacopla produtores e consumidores. O ganho, porém, não é proporcional, pois multiplicar K por 10 aumenta o *throughput* só 1,5 a 1,7 vezes. A partir daí o custo de processar cada item pesa mais que a espera.

Channel ou semáforos. A versão com channel é 1,5 a 1,6 vezes mais rápida em todos os K. Isso era esperado: o channel é implementado dentro do *runtime* e pode entregar o item direto a um consumidor que já está esperando, enquanto a versão com semáforos faz duas operações de semáforo e trava um mutex por item. A vantagem da versão com semáforos é mostrar cada passo do algoritmo clássico, não a velocidade. Como o consumo dela é mais caro, a fila também fica mais cheia (63 contra 54 em $K = 100$).

Distribuição entre consumidores. O consumidor 0, o do `time.After`, processa entre 44% e 99% da média dos quatro, porque cria um *timer* novo a cada volta do laço mesmo quando há item disponível. A diferença aumenta com K, já que os outros consumidores ficam livres para andar mais rápido. É um custo do requisito de timeout, e não *starvation*.

3.4 Encerramento ordenado

Na versão com channel, depois que o `WaitGroup` dos produtores termina, o canal é fechado. Os consumidores continuam retirando itens até o canal esvaziar, e então a leitura devolve `ok == false` e eles saem. Nenhum item se perde e ninguém envia em canal fechado.

Na versão com semáforos não dá para fechar os canais de permissões, porque eles não carregam itens. Então, depois dos produtores, inserimos uma sentinela por consumidor no próprio buffer. Como o buffer é FIFO e a produção já acabou, nenhum item fica atrás das sentinelas, e cada consumidor sai ao retirar a sua. Isso funciona mesmo com K menor que o número de consumidores, porque sempre sobra pelo menos um consumidor ativo para abrir espaço. No fim, o programa confere que o total produzido é igual ao consumido, que não há itens duplicados e que o buffer terminou vazio.

4 Validação

Os dois programas rodaram com `go run -race` sem nenhum aviso de *data race*. Nos filósofos testamos N de 2 a 100 nas duas estratégias e na versão `deadlock`, e uma cópia do programa que imprime `runtime.NumGoroutine()` no final mostrou que sempre sobra só a *goroutine* principal. No produtor/consumidor rodamos 100 configurações aleatórias (incluindo K menor que o número de consumidores, zero itens e produção lenta para forçar timeouts), e em todas o total produzido foi igual ao consumido. Os testes automáticos (`go test -race ./...`) também passam. No Windows, o `-race` precisa de um compilador C, e usamos o MinGW.

5 Uso de Ferramentas de IA

Usamos assistentes de IA em várias etapas, e todos os integrantes revisaram o resultado e conseguem explicar o código e as análises.

Organização. Um assistente de IA ajudou a montar a divisão de tarefas entre os quatro integrantes e o roteiro inicial de cada parte.

Problema 1. O Claude foi usado na implementação dos filósofos, na discussão das estratégias e das condições de Coffman e na depuração, incluindo o *watchdog* que detecta o *deadlock* e o sorteio dos tempos. Na análise de *fairness*, a IA foi usada como apoio na redação do texto sobre as condições de Coffman e a *starvation*. Na revisão final, o Claude foi usado para conferir o código com o enunciado, validar os parâmetros, coletar de novo os dados de *fairness* e atualizar este relatório.

Problema 2. A versão com channel e o consumo com timeout foram feitos com ajuda do Codex. A versão com semáforos, as medições e os experimentos foram feitos com ajuda do Claude. Nos dois casos a IA foi usada no projeto da solução, na geração do código e dos testes, na depuração, na análise dos dados e na redação das seções correspondentes.

6 Referências

1. DIJKSTRA, E. W. Hierarchical ordering of sequential processes. *Acta Informatica*, v. 1, n. 2, p. 115-138, 1971.
2. COFFMAN, E. G.; ELPHICK, M. J.; SHOSHANI, A. System deadlocks. *ACM Computing Surveys*, v. 3, n. 2, p. 67-78, 1971.
3. SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Fundamentos de Sistemas Operacionais*. 9. ed. Rio de Janeiro: LTC, 2015.
4. TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*. 4. ed. São Paulo: Pearson, 2016.
5. THE GO AUTHORS. *The Go Programming Language Specification*. Disponível em: `https://go.dev/ref/spec`.
6. THE GO AUTHORS. *The Go Memory Model*. Disponível em: `https://go.dev/ref/mem`.
7. THE GO AUTHORS. *Data Race Detector*. Disponível em: `https://go.dev/doc/articles/race_detector`.
8. THE GO AUTHORS. Documentação dos pacotes `sync`, `sync/atomic`, `time`, `math/rand` e `flag`. Disponível em: `https://pkg.go.dev/std`.
9. Material da disciplina de Fundamentos de Processamento Paralelo e Distribuído (PUCRS, 2026/2).